

Defect Log and Metrics

Clinic Booking System

Student Biruk Mulatu
Student ID [STUDENT ID — fill in before submission]
Instructor Abel Tadesse
Date 4 September 2026 (revised 6 September 2026)

1. Defect log

Five real defects were found and fixed during development — all through the project's own automated testing effort, none reported by an external user (there being none, this being a course project). DEF-001 to DEF-003 were found in the original course scope; DEF-004 and DEF-005 during the extension work. Each entry below follows the lifecycle New → In Progress → Fixed → Verified → Closed. Severity is impact if shipped as-is; priority is urgency relative to other work. The full log with every reproduction step is kept in `docs/defect-log.md`.

DEF-001 — Integration tests fail intermittently with a unique-constraint violation

Severity: Medium · **Priority:** High · **Status:** Closed · **Found:** Phase 6, first run of `AppointmentServiceIT` · **Component:** `integration/AppointmentServiceIT.java`

Steps to reproduce: Write `AppointmentServiceIT` as a plain `@SpringBootTest` (no `@Transactional`) with a `@BeforeEach` that inserts a `Patient` with a fixed username. Run `mvn clean verify`.

Expected: All test methods in the class pass independently.

Actual: The first method passes; every subsequent method fails:

```
org.springframework.dao.DataIntegrityViolationException: ... Unique index
or primary key violation: "PUBLIC.CONSTRAINT_INDEX_F ON
PUBLIC.PATIENT(USERNAME NULLS FIRST) VALUES ( / * 1 * / 'tigit' )"
```

Root cause: `@SpringBootTest` reuses one application context (and one H2 instance, via `DB_CLOSE_DELAY=-1`) across every method in the class. Without `@Transactional`, writes commit immediately and are never rolled back, so the second `@BeforeEach` collides with the first test's committed row.

Fix: Added class-level `@Transactional` to `AppointmentServiceIT` and `AppointmentJourneyIT`, so each method's writes roll back at the end of the method. Deliberately *not* applied to the Selenium `BookingJourneyIT`: Selenium drives the app over a real HTTP socket from a separate thread, so an open test-thread transaction would be invisible to the server thread — that class avoids the collision with a distinct seeded username per test instead.

Verification: `mvn clean verify` passes with all methods green, run repeatedly.

DEF-002 — CI coverage-summary step fails with a confusing secondary error whenever the build already failed

Severity: Low · **Priority:** Medium · **Status:** Closed · **Found:** during the deliberate regression demo (§3) · **Component:** `.github/workflows/ci.yml`, "Publish coverage summary" step

Steps to reproduce: Break a test in `et.aau.clinic.core` so `mvn clean verify` fails during the unit-test phase; push to a branch and let the GitHub Actions CI workflow run.

Expected: Only the real test failure is reported; the coverage-summary step is skipped or clearly reports "no coverage available".

Actual: The "Publish coverage summary" step (bound with `if: always()`) also fails, burying the real failure:

```
awk: fatal: cannot open file `target/site/jacoco/jacoco.csv' for reading: No such file or directory
Error: Process completed with exit code 2.
```

Root cause: `jacoco.csv` is produced by a plugin goal bound to Maven's `verify` phase. Maven's fail-fast behaviour stops the build at the first failing phase, so when unit tests fail in the earlier `test` phase, `verify` never runs and the CSV is never written — but the summary step

assumed it always would be.

Fix: Added a file-existence check at the top of the step: if `jacoco.csv` is missing, write a one-line "no coverage report was generated" note to the job summary and exit 0 instead of trying to parse a file that was never created.

Verification: Reproduced on the `regression-demo` branch (failing run 33305001970), fixed in the same commit that reverted the regression (passing run 33305212152) — see §3.

DEF-003 — Selenium system test is flaky under constrained CPU (Jenkins-in-Docker)

Severity: Medium · **Priority:** High · **Status:** Closed · **Found:** first real Jenkins pipeline run against `docker/jenkins/` · **Component:** `system/pages/*.java` (Page Objects)

Steps to reproduce: Run `BookingJourneyIT` in an environment with limited CPU relative to a GitHub-hosted runner (the Jenkins Docker container on a shared host). The page objects called `driver.findElement(...)` directly, immediately after a `.click()` that triggers a server-side redirect.

Expected: The system test passes reliably regardless of server response speed.

Actual:

```
org.openqa.selenium.NoSuchElementException: no such element: Unable to
locate element: {"method":"css selector","selector":"#confirmation-status"}
```

The same test passed reliably in GitHub Actions (hosted runner, more CPU) but failed the first time it ran inside the Jenkins container.

Root cause: Selenium's WebDriver spec only guarantees `click()` blocks until a triggered navigation *starts*, not until the resulting page has finished rendering. The page objects implicitly assumed the next page was already rendered — a race condition that held on a fast, lightly-loaded runner and broke under the container's tighter CPU budget.

Fix: Added `AbstractPage`, a common base class for all five page objects, with a `find(...)` helper backed by `WebDriverWait` + `ExpectedConditions.presenceOfElementLocated` (10s timeout). Every locator call now waits for its target element instead of assuming it is already there.

Verification: Reproduced in the Jenkins container (build #3 of `clinic-booking-pipeline`), fixed and verified locally, then re-run in the same container (build #4: Finished: SUCCESS) to confirm under the same constrained conditions that first exposed it.

The fix reduces the failure rate rather than proving the race eliminated - an explicit wait shortens the window, it does not remove it. As a data point: the same failure family recurred once on 2026-09-05 on a plain local `mvn clean verify` rerun under heavier-than-usual load, and was gone on immediate retry. Treated as mitigated, not closed against recurrence under sufficiently adverse timing.

DEF-004 — JSON API returned an HTML error page with HTTP 200 for a thrown business exception

Severity: Medium · **Priority:** High · **Status:** Closed · **Found:** adding the admin CRUD delete endpoints (`AdminCrudIT`) · **Component:** `web/api/ApiExceptionHandler.java` vs `web/GlobalExceptionHandler.java`

Steps to reproduce: Call `DELETE /api/slots/{id}` for a slot that has an appointment against it; the handler throws `IllegalStateException`.

Expected: 400 with a JSON `{ "message": "..."` body, via the API-scoped `ApiExceptionHandler`.

Actual: 200 with `Content-Type: text/html` and the body of `error.html` — which the React client's `fetch` wrapper cannot parse.

Root cause: Two `@ControllerAdvice` beans both declared `@ExceptionHandler` for `IllegalStateException` / `IllegalArgumentException`: the API-scoped `ApiExceptionHandler` and the global `GlobalExceptionHandler` (which returns the view name "error"). Neither carried an `@Order`, so both sat at `LOWEST_PRECEDENCE` and the ordering between them was undefined — in this build the global one won for the new `DELETE` handlers. This is a genuine defect in **application code**, not test infrastructure.

Fix: Added `@Order(Ordered.HIGHEST_PRECEDENCE)` to `ApiExceptionHandler`. It is package-scoped to `et.aau.clinic.web.api`, so making it highest precedence cannot affect the Thymeleaf controllers.

Verification: `AdminCrudIT` (11 tests) — the guarded deletes now return 400 with a JSON message; `mvn clean verify` green.

DEF-005 — Concurrent bookings of the same slot both succeed (slot double-booked)

Severity: High · **Priority:** Medium · **Status:** Closed · **Found:** deliberately, adding a concurrency test for Rule 2's C1 · **Component:** service/AppointmentService , repository/SlotRepository

Steps to reproduce: BookingConcurrencyIT — 8 patients call requestBooking for one slot from 8 threads released together by a CountDownLatch .

Expected: Exactly one booking approved; the rest rejected SLOT_UNAVAILABLE . Exactly one active appointment for the slot afterwards.

Actual (pre-fix): 2–4 threads (timing-dependent) read the slot as free before any had committed its insert, so multiple REQUESTED rows were created for the one slot. Confirmed by reverting the fix: the test fails with expected: 1 on the approved count.

Root cause: Classic check-then-act race. requestBooking read "is the slot free?" and then inserted the appointment as two separate statements with no lock spanning them, and the method was not transactional. This defect is in **application code**, and it is exactly the "multi-user concurrency" case the original test plan had listed as out of scope — the concurrency test brought it into scope and found it.

Fix: A @Lock(PESSIMISTIC_WRITE) query (SlotRepository.findByIdForUpdate) plus @Transactional on requestBooking , bookForPatient and reschedule . The row-level write lock on the slot serialises the check-then-insert section, so the second caller blocks until the first commits, then sees its appointment and is rejected normally.

Verification: BookingConcurrencyIT passes with the fix and fails without it (verified by reverting). Full mvn clean verify green; all other tests unaffected.

2. Metrics

All figures recomputed from a fresh mvn clean verify + pitest:mutationCoverage run on 2026-09-06, on the merged main .

Metric	Value	Interpretation
Defect density	5 defects / $\approx$ 9.9 KLOC delivered (5,138 lines production code + 4,778 lines test code) $\approx$ 0.5 defects/KLOC	2 of the 5 defects (DEF-004, DEF-005) are in application code; the other 3 are in test infrastructure (DEF-001, DEF-003) or CI scripting (DEF-002). Against production code alone the figure is $\approx$ 0.4/KLOC. Both application defects are in the extension work, not the original course rules — the four course rules, derived test-first from formal techniques, have had no defect found in them at any point. Density fell from the earlier 1.4/KLOC mainly because the codebase roughly quadrupled while the defect count grew by two.
Defect removal efficiency (DRE)	5 found internally / (5 found + 0 escaped) = 100%	Every defect was caught by the project's own automated suite or its CI runs, before any external party exercised the system. DEF-005 in particular was found only because a concurrency test was written specifically to probe a gap the test plan had flagged. This figure is 100% only because the project has not yet passed an external evaluation gate — see the residual-risk discussion in the test summary report.
Defects found vs. escaped	5 found, 0 escaped (as of 2026-09-06, pre-submission)	"Escaped" only becomes a meaningful category once the instructor's evaluation runs.
Branch coverage — et.aau.clinic.core (the graded target)	100% (134/134 branches), against an 80% gate enforced by jacoco:check	Every branch in all 10 pure business-rule classes (the four course rules plus F–L) is exercised — a direct consequence of deriving tests systematically from EP/BVA/decision-table/state-transition/pairwise analysis (Part B) rather than writing tests ad hoc. One branch (a null-password guard in Rule F) surfaced uncovered during the branch merge and was closed with a dedicated case (TC-R10b).
Mutation score — et.aau.clinic.core (PIT)	100% (165/165 mutants killed), against a 90% gate enforced by PIT's mutationThreshold	Stronger evidence than branch coverage: PIT injected 165 mutations across the 10 rule classes (flipping $<$ to $\leq$, changing threshold constants, replacing return values, negating conditions) and <i>every one</i> was detected by a failing test. This shows the boundary and decision-table cases are derived tightly enough that a small change to the logic cannot pass unnoticed. See docs/mutation-testing.md .
Branch coverage — whole application (excluding domain , config , ClinicApplication , per the JaCoCo exclusions in pom.xml)	72.5% branch (332/458); instruction 81.8% (4,416/5,398); line 83.5% (947/1,134) — across 83 analysed classes	The gate only requires core/ , which is at 100%. The whole-application figure is held down almost entirely by the JSON API package (web/api/ : the *ApiController classes and their DTOs), which is additive, ungraded code for the separate React front end and has no dedicated unit tests — it is exercised only indirectly by the MockMvc integration tests. The pre-existing web/ (Thymeleaf controllers), service/ and repository/ packages remain fully branch-covered. See §3 of the test summary report for what is deliberately left uncovered and why.

3. Regression demonstration (context for the defect/CI story)

A deliberate regression was introduced to prove the pipeline actually catches breakage, not just that tests exist. Branch `regression-demo`, commit `9b6c9fe`, shifted Rule 1's CHILD/ADULT boundary from `age ≤ 17` to `age ≤ 16` — a one-line off-by-one that would silently overcharge every 17-year-old patient (ADULT fee, 250 ETB, instead of CHILD, 100 ETB) if it reached production.

	Commit	CI run	Result
Before (regression introduced)	<code>9b6c9fe</code>	run 33305001970	Failure (22s) — caught by <code>fee_atUpperBoundaryOfChildBand_age17_is100</code> , written in Phase 3, before this regression existed
After (regression reverted + DEF-002 fixed)	<code>4a98ab9</code>	run 33305212152	Success (52s) — full suite green (58 unit + 9 integration + 2 system <i>at that time</i>), coverage gate passed

Full narrative in `docs/regression-demo.md`. This is exactly what boundary value analysis is for: the test exists at age 17 specifically because that is the boundary, and a one-line shift of the boundary was caught immediately by the one test written to sit on it. The suite counts in the "After" row are the branch's state on 2026-09-05; the current `main` suite is 228 unit + 72 integration + 5 system (see the test summary report), and the same `fee_atUpperBoundaryOfChildBand_age17_is100` test still guards this boundary.